

Universidade de São Paulo
Instituto de Ciências Matemáticas e de Computação

SSC0903 — Computação de Alto Desempenho
Primeiro Trabalho Prático

Incêndio Florestal

Implementações sequencial e paralela com OpenMP

Integrante	Número USP
João Ricardo de Almeida Lustosa	15453697
Catarina Moreira Lima	8957221
Larissa Pires Moreira Rocha Duarte	15522358
Gabriel Phelippe Prado	15453730
Frederico Scheffel Oliveira	15452718

São Carlos
Setembro de 2026

Sumário

1	Introdução	3
2	Objetivos	3
2.1	Objetivo geral	3
2.2	Objetivos específicos	3
3	Modelagem do problema	4
3.1	Domínio espacial e temporal	4
3.2	Parâmetros e restrições da entrada	4
3.3	Cobertura, umidade e inicialização determinística	4
3.4	Estados e estruturas de armazenamento	5
3.5	Zonas de contenção	5
3.6	Vizinhança e potencial de ignição	6
3.7	Transições, atualização síncrona e parada	6
3.8	Estatísticas e saída	7
4	Implementação sequencial	7
4.1	Organização do código	7
4.2	Preparação da simulação	8
4.3	Laço principal	9
4.4	Finalização e custo	9
5	Implementação paralela	10
5.1	Estratégia de paralelização	10
5.2	Compartilhamento e sincronização	11
5.3	Escalonamento	12
6	Validação da correção	12
7	Ambiente e metodologia experimental	13
7.1	Ambiente das duas versões	13
7.2	Cargas e protocolo de execução	14
7.3	Delimitação do trecho cronometrado	15
7.4	Métricas	15
8	Resultados	15
8.1	MacOs	15
8.2	Linux	18
8.3	Efeito da vetorização automática (SIMD) na linha de base sequencial . . .	22

1 Introdução

Simulações em grades discretas permitem representar a evolução de um sistema por meio de regras locais aplicadas repetidamente a suas células. Neste trabalho, uma matriz retangular representa uma área florestal na qual o fogo se propaga a partir de focos iniciais. A evolução depende da cobertura e da umidade de cada célula, da presença de vizinhos em chamas, da direção e da intensidade do vento e da ativação de zonas de contenção em instantes predeterminados [1].

O modelo utiliza passos discretos e atualização síncrona: o resultado de uma célula em um passo é calculado a partir da configuração atual, sem utilizar resultados parcialmente produzidos para outras células. Essa propriedade permite distribuir as atualizações entre threads, desde que sejam respeitadas as dependências entre as etapas. A ativação das contenções deve terminar antes da propagação, e a atualização completa deve preceder a troca das estruturas que representam as configurações atual e seguinte.

A atividade relaciona, assim, correção e desempenho em programação paralela com memória compartilhada. A distribuição das células pode reduzir o tempo de processamento, mas introduz custos de sincronização e escalonamento. Além disso, o trabalho realizado por uma célula depende de seu estado: células intactas avaliam a vizinhança, enquanto outros estados exigem transições mais simples. A quantidade de threads, a política de distribuição das iterações e o tamanho da grade são, portanto, aspectos relevantes para a avaliação experimental.

O projeto contempla duas implementações em C: uma sequencial, utilizada como referência de comparação, e outra paralela com OpenMP. Ambas devem produzir os mesmos resultados para uma mesma entrada, exceto pelo tempo de execução. Embora a inicialização empregue geração pseudoaleatória, a semente e a ordem das chamadas são fixadas, e não há decisões aleatórias durante a propagação. O modelo é o definido para a disciplina e não pretende representar um modelo físico validado de incêndios florestais.

2 Objetivos

2.1 Objetivo geral

Implementar e avaliar uma simulação determinística da propagação de incêndio, comparando uma solução sequencial em C com uma solução paralela em C/OpenMP, de modo a preservar as regras do modelo e investigar o desempenho em diferentes configurações de execução.

2.2 Objetivos específicos

- Representar a floresta em armazenamento linear e implementar a leitura, validação e inicialização dos dados conforme o enunciado.
- Implementar as transições de estado, a influência direcional do vento, os tempos de queima e a ativação de zonas de contenção.
- Desenvolver uma versão sequencial que sirva de referência para a validação da versão paralela, complementada por testes das regras do modelo.

- Distribuir a ativação e a atualização das células em uma região paralela persistente, utilizando `omp for`, SIMD, reduções e sincronização adequada.
- Verificar a equivalência das saídas entre versões e a independência dos resultados em relação à quantidade de threads e ao schedule.
- Comparar pelo menos duas políticas de escalonamento e analisar tempos, speedup e eficiência em diferentes cargas e quantidades de threads.

3 Modelagem do problema

3.1 Domínio espacial e temporal

A área é representada pelo conjunto de células

$$\Omega = \{(l, c) \mid 0 \leq l < L, 0 \leq c < C\}, \quad (1)$$

com $N = LC$ posições. O armazenamento é linear, com índice $i = lC + c$. As coordenadas podem ser recuperadas pela divisão inteira $l = i/C$ e pelo resto $c = i \bmod C$. Essa representação permite percorrer a grade por um único laço e distribuir suas iterações entre threads.

A evolução ocorre em, no máximo, P passos, numerados de 0 a $P - 1$. Uma configuração inicial é construída antes do primeiro passo. Em cada iteração, a aplicação das regras produz a configuração seguinte, que se torna a base da próxima atualização.

3.2 Parâmetros e restrições da entrada

O arquivo informa L, C, P , a quantidade de threads T , a semente e o limiar de ignição. Em seguida, especifica as componentes do vento e sua intensidade, as quantidades de focos (F) e de zonas (Z), as coordenadas dos focos e os retângulos de contenção com seus passos de ativação.

São exigidos $L, C, T, \text{LIMIAR} > 0, P, F, Z \geq 0$, componentes do vento em $\{-1, 0, 1\}$, direção diferente de $(0, 0)$ e intensidade entre 0 e 5. Os focos devem ser distintos, internos à grade e posicionados sobre células combustíveis. As zonas devem ter limites ordenados e internos à grade, com $0 \leq p_{\text{ativação}} < P$. Consequentemente, não há passo de ativação válido quando $P = 0$.

As duas aplicações recebem somente o caminho do arquivo de entrada:

```
./fire_seq entrada.txt
./fire_omp entrada.txt
```

O valor de T é obtido desse arquivo, inclusive nos experimentos que variam a quantidade de threads.

3.3 Cobertura, unidade e inicialização determinística

A cobertura e a unidade são propriedades fixas de cada célula. A inicialização percorre a matriz em ordem crescente de linhas e colunas, com uma única thread. Para cada

posição, primeiro obtém-se `rand_r(&seed) % 100` para determinar a cobertura e, imediatamente depois, `rand_r(&seed) % 101` para determinar a umidade. Preservar essa ordem é necessário para que as versões recebam a mesma configuração inicial no ambiente experimental.

Tabela 1: Coberturas e parâmetros do modelo. As proporções são as da regra de geração, não contagens exatas de cada matriz.

Código	Cobertura	Sorteio	Fator	Queima
0	Água	0–9	0	—
1	Solo exposto	10–19	0	—
2	Vegetação rasteira	20–54	8	2 passos
3	Floresta	55–99	12	4 passos

Água e solo exposto começam no estado não combustível. Vegetação rasteira e floresta começam intactas, exceto nas posições dos focos iniciais, que passam ao estado em chamas e recebem o tempo de queima de sua cobertura. A quantidade de células inicialmente combustíveis é registrada antes da evolução, incluindo os focos, para servir de denominador dos percentuais finais.

3.4 Estados e estruturas de armazenamento

O estado de uma célula pode ser não combustível (0), intacta (1), em chamas (2), queimada (3) ou contenção (4). Cobertura e estado são conceitos distintos: a cobertura identifica o material da célula; o estado indica sua situação na evolução do incêndio.

Tabela 2: Vetores utilizados para representar a floresta.

Vetor	Função
<code>cobertura</code>	Tipo de cobertura de cada posição.
<code>umidade</code>	Umidade inteira entre 0 e 100.
<code>estado_atual</code>	Configuração usada como base da atualização.
<code>proximo_estado</code>	Configuração produzida pelo passo.
<code>tempo_atual</code>	Tempo restante de queima na configuração atual.
<code>proximo_tempo</code>	Tempo restante calculado para a configuração seguinte.
<code>ativacao</code>	Primeiro passo de contenção da posição, ou -1 .

Os dois pares de vetores de estado e tempo evitam que uma célula utilize uma atualização parcial de sua vizinha. Ao fim de cada passo, os ponteiros de cada par são trocados. O armazenamento da grade é $O(LC)$; somadas as descrições dos focos e zonas, o espaço é $O(LC + F + Z)$.

3.5 Zonas de contenção

Cada zona corresponde a um retângulo de limites inclusivos e a um passo de ativação. Antes da simulação, constrói-se um mapa com o menor passo associado a cada célula nas

zonas sobrepostas; posições sem zona recebem -1 . A construção desse mapa não integra o trecho cronometrado.

No início do passo programado, apenas células intactas se tornam contenção. Os demais estados permanecem inalterados: uma zona não apaga fogo existente nem recupera células queimadas. A ativação ocorre somente no passo indicado, e as células que se tornam contenção permanecem nesse estado.

3.6 Vizinhaça e potencial de ignição

A propagação utiliza os oito vizinhos de Moore, ignorando posições fora da grade. Apenas vizinhos em chamas contribuem. Para uma célula-alvo (l, c) e um vizinho (l_v, c_v) , define-se a direção do vizinho para o alvo:

$$\Delta l = l - l_v, \quad \Delta c = c - c_v. \quad (2)$$

O peso básico b_v vale 10 se $|\Delta l| + |\Delta c| = 1$ e 7 para vizinhos diagonais. Dadas a direção fixa (w_l, w_c) e a intensidade V do vento, o alinhamento e o peso ajustado são

$$A_v = \Delta l w_l + \Delta c w_c, \quad (3)$$

$$P_v = \max(1, b_v + V A_v). \quad (4)$$

O produto escalar pode assumir valores entre -2 e 2 . Valores positivos favorecem a propagação; negativos a desfavorecem. Quando $V = 0$, o peso ajustado coincide com o básico.

Se f_i é o fator combustível e u_i a unidade da célula-alvo, seu potencial é

$$S_i = \sum_{v \text{ em chamas}} P_v, \quad I_i = \left\lfloor \frac{S_i f_i (100 - u_i)}{100} \right\rfloor. \quad (5)$$

Uma célula intacta se incendeia quando $I_i \geq \text{LIMIAR}$. Todos esses cálculos usam aritmética inteira, evitando diferenças de arredondamento na propagação entre as versões.

3.7 Transições, atualização síncrona e parada

Após a ativação das zonas, células não combustíveis, queimadas e de contenção mantêm seus estados. Células intactas permanecem intactas ou se incendeiam conforme o potencial. Células já em chamas têm seu tempo reduzido em uma unidade e tornam-se queimadas ao atingir zero.

Uma célula recém-incendiada recebe o tempo completo de sua cobertura. Ela não propaga fogo imediatamente dentro do mesmo passo: sua contribuição passa a existir quando a configuração produzida se torna a configuração atual. Essa separação preserva a atualização síncrona e permite que as células sejam processadas em ordens diferentes sem alterar o resultado.

A sequência definida para cada passo é: ativar as zonas, produzir os próximos estados e tempos, calcular as estatísticas do próximo estado, trocar os vetores e verificar a parada. A execução termina ao completar P passos ou quando não há células em chamas. Se não houver fogo após a inicialização, nenhum passo é executado, mesmo com contenções futuras.

3.8 Estatísticas e saída

As novas ignições de um passo são as transições de intacta para em chamas; os focos iniciais não integram esse total. O pico registra a maior quantidade de novas ignições e, em empate, o primeiro passo. Na ausência de novas ignições, o resultado do pico é $(-1, 0)$.

Se B_0 é a quantidade de células inicialmente combustíveis, Q a quantidade final queimada, H a quantidade final em chamas e K a quantidade final em contenção, os percentuais são

$$\text{Queimado} = 100 \frac{Q + H}{B_0}, \quad \text{Protegido} = 100 \frac{K}{B_0}. \quad (6)$$

Quando $B_0 = 0$, ambos são zero. Células ainda em chamas entram no percentual queimado porque já foram atingidas pelo incêndio.

O checksum é calculado sequencialmente após a simulação, na ordem linear da matriz. Partindo de zero, para cada célula incorpora-se primeiro o estado e depois o tempo, pela operação $h \leftarrow 31h + x$, utilizando `unsigned long long` conforme o enunciado. Ele complementa a comparação dos resultados, sem substituir a verificação das demais saídas.

A saída contém o número de passos executados, as cinco contagens finais, o total e o pico de ignições, os dois percentuais, o checksum e o tempo. Os percentuais têm duas casas decimais e o tempo tem seis; todos os campos, exceto o tempo, devem coincidir entre as versões.

4 Implementação sequencial

A versão sequencial, em `fire_seq.c`, é a referência de correção e de desempenho do trabalho. Ela segue diretamente a modelagem da Seção 3, sem otimizações que alterem a ordem das operações, de modo que cada etapa do passo corresponde a uma função com responsabilidade única.

4.1 Organização do código

Os parâmetros de entrada, os focos, as zonas e os vetores da Tabela 2 são agrupados na estrutura `Simulacao`, passada por ponteiro a todas as funções. Focos e zonas são representados pelas estruturas `Foco` e `ZonaContencao`. A Tabela 3 resume as funções e indica quais pertencem ao trecho cronometrado.

Tabela 3: Funções da versão sequencial.

Função	Responsabilidade	Cronometrada
<code>ler_entrada</code>	Leitura e validação do arquivo e dos limites de tamanho.	Não
<code>precalcular_pesos</code>	Pré-cálculo do peso P_v de cada uma das 8 direções de Moore.	Não
<code>alocar_estruturas</code>	Alocação dos sete vetores de tamanho LC .	Não
<code>gerar_floresta</code>	Sorteio da cobertura e da umidade; estado inicial.	Não
<code>aplicar_focos</code>	Marcação dos focos e validação da cobertura.	Não
<code>construir_mapa_ativacao</code>	Menor passo de ativação por célula.	Não
<code>ativar_zonas</code>	Conversão de intactas em contenção no passo.	Sim
<code>potencial_ignicao</code>	Cálculo de I_i para uma célula intacta.	Sim
<code>calcular_proximo_estado</code>	Transições e contagem de ignições.	Sim
<code>trocar_matrizes</code>	Troca dos ponteiros atual/próximo.	Sim
<code>existe_em_chamas</code>	Verificação da condição de parada.	Sim
<code>calcular_checksum</code>	Checksum na ordem linear.	Não

4.2 Preparação da simulação

A leitura verifica o retorno de cada `fscanf` e aplica as restrições da Seção 3.2, encerrando o programa com `EXIT_FAILURE` diante de qualquer inconsistência. A verificação de que os focos estão sobre células combustíveis só é possível após a geração da floresta e, por isso, é feita em `aplicar_focos`. L , C e as coordenadas de focos e zonas continuam do tipo `int`, e a leitura rejeita grades em que LC excede `INT_MAX` ou em que o tamanho em bytes dos vetores não cabe em `size_t`; verificação análoga é aplicada às quantidades de focos e zonas. Já os laços que percorrem a grade inteira (por exemplo, `construir_mapa_ativacao`, `contar_estados_finais` e o índice linear usado em `potencial_ignicao/atualizar_celula`) foram padronizados para `long long`, com o produto $L \times C$ calculado com um cast explícito; isso elimina, para grades maiores que as testadas neste relatório, o risco de estouro que já era tratado à parte em `calcular_checksum`. Todas as alocações têm o retorno conferido.

Logo após `ler_entrada` e ainda fora do trecho cronometrado, `precalcular_pesos` calcula, uma única vez, o peso P_v de cada uma das 8 direções de Moore (Seção 3.7 adiante) e os armazena em `sim->pesos[8]`. Como P_v depende apenas da direção fixa e dos parâmetros de vento e intensidade lidos da entrada — nunca do estado da simulação — calculá-lo uma vez evita repetir a mesma conta para cada vizinho, de cada célula intacta, em cada passo, tanto na versão sequencial quanto na paralela.

Na geração da floresta, a semente lida é copiada para uma variável local do tipo `unsigned int`, tipo exigido por `rand_r`, evitando a conversão de ponteiros entre `int` e `unsigned int`. As chamadas passam pela função `fire_rand_r`, que utiliza o `rand_r` nativo no Linux e, no Windows e no macOS, uma reimplementação do algoritmo da glibc. No Windows, `rand_r` não existe; no macOS, existe, mas emprega outro algoritmo e

produziria uma floresta diferente para a mesma semente. Assim, a configuração inicial é a mesma nas três plataformas. A ordem das chamadas é a descrita na Seção 3.3: primeiro a cobertura e, em seguida, a umidade da mesma célula.

O mapa de ativação é inicializado com -1 e preenchido percorrendo cada retângulo; em zonas sobrepostas, prevalece o menor passo. Por fim, a quantidade de células inicialmente combustíveis B_0 é contada a partir da cobertura, antes do início da cronometragem.

4.3 Laço principal

O trecho cronometrado é delimitado por duas chamadas a `omp_get_wtime()`; o cabeçalho `omp.h` é incluído apenas para essa medição, e o programa não contém diretivas OpenMP. O laço segue a ordem definida na Seção 3.7:

```
se existe_em_chamas():
    para passo = 0 ate P-1:
        ativar_zonas(passo)
        ignicoes = calcular_proximo_estado()
        total += ignicoes
        se ignicoes > pico_qtd: pico = (passo, ignicoes)
        trocar_matrizes()
        se nao existe_em_chamas(): interromper
```

A função `ativar_zonas` percorre toda a grade e altera `estado_atual` diretamente, o que é seguro porque ocorre antes da propagação do mesmo passo. Em `calcular_proximo_estado`, cada célula é tratada conforme seu estado: não combustíveis, queimadas e contenções são copiadas com tempo zero; células em chamas têm o tempo decrementado e passam a queimadas ao atingir zero; células intactas chamam `potencial_ignicao`. Todas as leituras são feitas em `estado_atual` e `tempo_atual`, e todas as escritas em `proximo_estado` e `proximo_tempo`, o que garante a atualização síncrona.

O potencial percorre os 8 deslocamentos fixos de Moore, na mesma ordem usada por `precalcular_pesos`, e descarta vizinhos fora da grade. Para cada vizinho em chamas, o peso P_v já vem pronto de `sim->pesos[k]`, sem recalculá-lo nem o alinhamento A_v a cada chamada. O resultado final usa divisão inteira, equivalente ao piso para valores não negativos.

O pico de ignições é atualizado apenas quando a contagem do passo é estritamente maior que o máximo anterior, o que mantém o primeiro passo em caso de empate. Como `pico_passo` começa em -1 e `pico_qtd` em 0, o resultado $(-1, 0)$ é preservado quando não há novas ignições. A troca das configurações é feita apenas pela troca dos ponteiros, sem cópia de dados.

4.4 Finalização e custo

Após a cronometragem, são contados os estados finais, calculados os percentuais com B_0 como denominador e obtido o checksum, acumulado em `unsigned long long`, cujo estouro é aritmética modular bem definida em C. A memória é liberada ao final.

Cada passo realiza três varreduras da grade: ativação das zonas, cálculo do próximo estado e verificação de parada, esta última interrompida no primeiro foco encontrado.

Como cada célula intacta examina no máximo oito vizinhos, o custo por passo é $O(LC)$ e o custo total do trecho cronometrado é $O(PLC)$. O trabalho por célula, porém, não é uniforme: células intactas são mais caras que as demais, o que é relevante para a escolha do escalonamento na versão paralela.

5 Implementação paralela

A versão paralela, em `fire_omp.c`, parte do código sequencial e mantém idênticas a leitura, a validação, a alocação, a geração da floresta, a aplicação dos focos, a construção do mapa de ativação e o cálculo dos resultados finais. Todas essas etapas estão fora do trecho cronometrado, e a geração da floresta deve, por exigência do enunciado, ser feita por uma única thread. A paralelização concentra-se, portanto, no laço de passos, encapsulado na função `simular`.

5.1 Estratégia de paralelização

Como a atualização é síncrona, o próximo estado de cada célula depende apenas de `estado_atual`, `tempo_atual` e dos dados fixos, e cada iteração escreve somente na própria posição de `proximo_estado` e `proximo_tempo`. Não há, assim, dependência entre iterações de um mesmo passo, e o laço sobre as células pode ser dividido entre threads por decomposição de domínio sobre o índice linear. Entre passos, por outro lado, a dependência é total: o passo $p + 1$ só pode começar depois que toda a configuração do passo p estiver pronta.

Em vez de abrir uma região paralela a cada passo, o programa utiliza uma única região `parallel` persistente, que envolve todo o laço `while`. As threads são criadas uma vez e percorrem juntas os passos, o que evita o custo repetido de criação e sincronização da equipe ao longo de até P passos. Dentro dela, cada passo é organizado da seguinte forma:

```
#pragma omp parallel num_threads(T)
while (continua):
    #pragma omp for schedule(guided, 1024) \
        reduction(+:ignicoes) reduction(||:tem_chamas)
        // ativacao da zona (inlineada) + atualizar_celula
    #pragma omp single                                // estatisticas, troca, parada
```

Em uma versão anterior, a ativação das zonas era feita por um `omp for` órfão dedicado (`ativar_zonas`), com sua própria barreira implícita antes do laço principal. Essa barreira foi eliminada: a checagem de ativação (`if (sim->ativacao[idx]==passo && sim->estado_atual[idx]==1) sim->estado_atual[idx]=4;`) passou a ser inlineada no topo do corpo do `omp for` principal, imediatamente antes da chamada a `atualizar_celula` para o mesmo `idx`. Isso é seguro porque a ativação só transiciona INTACTA (1) para CONTENÇÃO (4) e nunca toca em EM CHAMAS (2), enquanto `potencial_ignicao` só se importa se o vizinho está em chammas: uma thread que leia `estado_atual` de um vizinho antes de ele ser ativado no mesmo passo o vê como 1, e depois de ativado o vê como 4, mas nunca como 2 – logo o valor de S_i não muda, independentemente de qual das duas leituras ocorra. A ordem entre ativação e cálculo de potencial de ignição é, portanto, indiferente ao resultado, o que permite eliminar uma barreira inteira por passo (de 3 para 2) sem alterar o checksum nem a contagem de passos (Seção 6).

A atualização das células foi reorganizada na função `atualizar_celula`, que processa uma única posição. Além do indicador de nova ignição, a função informa se a célula estará em chamas no próximo estado. Esses valores são combinados por duas reduções: soma para as novas ignições e disjunção lógica para a existência de fogo. Com isso, a condição de parada passa a ser obtida no próprio laço de atualização, e a varredura adicional de `existe_em_chamas` a cada passo, presente na versão sequencial, deixa de ser necessária. A função continua sendo usada apenas na verificação inicial, antes da região paralela; se não houver fogo ou se $P = 0$, a região nem chega a ser aberta.

Em `potencial_ignicao`, os oito deslocamentos de Moore são armazenados em vetores constantes de escopo de arquivo (DL/DC, compartilhados com `precalcular_pesos`), e o laço sobre eles foi reescrito sem desvios (`continue`), para permitir a vetorização com `omp simd reduction(+:S)`. Um vizinho fora da grade tem o índice substituído pelo da própria célula, sempre válido, e sua contribuição é anulada pelo indicador `dentro`; o mesmo ocorre com vizinhos que não estão em chamas. Assim como na versão sequencial, o peso P_v de cada direção vem pronto de `sim->pesos[k]`, e não é recalculado a cada vizinho – aqui esse ganho é ainda mais relevante, pois evita repetir a mesma conta escalar dentro do laço vetorizado. A soma S_i resultante é a mesma da versão sequencial.

5.2 Compartilhamento e sincronização

A região paralela usa `default(none)`, o que obriga a declarar explicitamente o escopo de cada variável. São compartilhados o ponteiro para a estrutura `Simulacao`, o tamanho n e as variáveis de controle do laço: `passo`, `continua`, o total e o pico de ignições e os dois acumuladores das reduções. O índice dos laços `omp for` e as variáveis locais de `atualizar_celula` e `potencial_ignicao` são privados de cada thread. Os vetores da simulação são compartilhados, mas nenhuma posição é escrita por mais de uma thread no mesmo laço.

A corretude depende das barreiras implícitas ao final de cada construção de compartilhamento de trabalho. Após a fusão descrita acima, restam apenas duas por passo, e não mais três:

- a barreira ao final do `omp for` principal garante que todos os próximos estados estejam escritos e que as reduções estejam combinadas antes do bloco `single` – a ativação inlineada no início do mesmo laço não precisa de barreira própria, pelo argumento de correção da Seção anterior;
- a barreira ao final do `single` garante que a troca de ponteiros e os novos valores de `passo` e `continua` sejam vistos por todas as threads antes da próxima iteração do `while`.

As operações de custo constante do passo, como a atualização do total e do pico, a troca dos vetores, o incremento de `passo` e o cálculo de `continua`, são feitas por uma única thread no bloco `single`. O mesmo bloco zera os acumuladores das reduções para o passo seguinte, o que dispensa um segundo `single` e uma barreira adicional por passo. Não há uso de `critical` nem de `atomic`: toda a combinação de resultados parciais é feita pelas reduções.

O número de threads é definido por `num_threads(T)`, com o valor lido do arquivo de entrada, e `omp_set_dynamic(0)` é chamado antes da região para impedir que o ambiente

reduza o tamanho da equipe. Como as reduções operam sobre inteiros, cuja soma é associativa, os resultados não dependem da ordem em que as threads combinam seus valores parciais. As saídas são, portanto, as mesmas para qualquer quantidade de threads e qualquer política de escalonamento.

5.3 Escalonamento

Existe um único laço `omp for` por passo, e ele governa tanto a ativação inlineada quanto a atualização da célula; a política de escalonamento escolhida precisa, portanto, equilibrar o custo do laço como um todo. Esse custo por iteração é irregular: a comparação de ativação é praticamente uniforme, mas `atualizar_celula` não é. As células intactas calculam o potencial sobre oito vizinhos, enquanto as demais apenas copiam ou decrementam valores. A distribuição desse custo muda ao longo da simulação, à medida que a frente de fogo avança e as regiões queimadas e contidas crescem. Uma divisão estática em blocos iguais poderia atribuir a uma thread uma faixa com muito mais células intactas que a de outra. Por isso, o laço usa `schedule(guided, 1024)`: os blocos começam grandes e diminuem conforme o trabalho restante, equilibrando a carga no final do laço. O tamanho mínimo de 1024 iterações limita a quantidade de requisições ao escalonador e preserva a localidade, evitando a sobrecarga de políticas como `dynamic` com blocos unitários. A política é declarada explicitamente porque, sem a cláusula, o escalonamento adotado depende da implementação.

A comparação entre esta política e outras alternativas, exigida pelo enunciado, é apresentada na Seção 8.

6 Validação da correção

Tabela 4: Validação da versão final (macOS), após a fusão de barreiras, o pré-cálculo dos pesos e a padronização dos índices para `long long`, nas três políticas de escalonamento comparadas.

Caso	Threads	Schedule	Resultado
Carga pequena	2, 4, 6, 8	guided(1024), static, dynamic(1024)	Checksum 17073017104646724864; 80 passos
Carga média	2, 4, 6, 8	guided(1024), static, dynamic(1024)	Checksum 17187382406529706254; 100 passos
Carga grande	2, 4, 6, 8	guided(1024), static, dynamic(1024)	Checksum 14825089465781038537; 100 passos

Para cada carga, o checksum e a quantidade de passos executados coincidiram nas 150 execuções (10 sequenciais e 4×10 OpenMP por política, nas três políticas de escalonamento, com $T \in \{2, 4, 6, 8\}$), confirmando que o resultado independe da quantidade de threads e da política de escalonamento, conforme exigido pelo enunciado.

Nas três cargas a simulação consumiu todos os P passos configurados, isto é, o incêndio não se extinguiu antes do limite definido na Tabela de cargas. As entradas com zero passos, sem focos e com zonas de contenção sobrepostas foram contempladas na validação da entrada (Seção 3.2) e na lógica de `construir_mapa_ativacao`.

7 Ambiente e metodologia experimental

7.1 Ambiente das duas versões

Tabela 5: Ambiente experimental — macOS.

Item	Configuração
CPU e núcleos	Apple M4 (4 núcleos de desempenho + 6 de eficiência; 10 núcleos no total, sem SMT/hyperthreading)
Memória e sistema	16 GiB de RAM; macOS 26.5.2 (build 25F84); arquitetura arm64 (Apple Silicon)
Compilador	Apple clang 17.0.0 (clang-1700.6.4.2), target=arm64-apple-darwin25.5.0, com libomp do Homebrew (/opt/homebrew/opt/libomp)
Compilação sequencial	Makefile.macos, alvo <code>fire_seq</code> : <code>clang -O3 -Wall -Wextra -std=c11 -Xpreprocessor -fopenmp -I<libomp>/include -L<libomp>/lib -lomp fire_seq.c -o fire_seq</code>
Compilação paralela	mesmos parâmetros de compilação (inclusive <code>-O3</code>), alvo <code>fire_omp</code> (<code>fire_omp.c</code>) – os dois binários recebem o mesmo nível de otimização, incluindo a vetorização automática (SIMD) que o compilador aplica a ambos
Execução OpenMP	<code>T</code> lido do arquivo de entrada e fixado com <code>num_threads(T)</code> e <code>omp_set_dynamic(0)</code> ; a política de escalonamento do laço de atualização é fixada em tempo de compilação na diretiva <code>omp for</code> – três políticas foram medidas nesta rodada (<code>guided(1024)</code> , <code>static</code> e <code>dynamic(1024)</code>), cada uma recompilando <code>fire_omp</code> isoladamente; a versão final do código usa <code>guided(1024)</code>
Versão dos fontes	branch <code>main</code> , com a fusão de barreiras, o pré-cálculo dos pesos dos vizinhos e a padronização dos índices para <code>long long</code> descritos na Seção 3.7 e adiante (commit a ser registrado após esta rodada de otimizações)

Tabela 6: Ambiente experimental — Linux.

Item	Configuração
CPU e núcleos	AMD Ryzen 5 (8 núcleos no total)
Memória e sistema	8 GiB de RAM; Ubuntu 24.04.4 LTS; arquitetura x86_64
Compilador	gcc (Ubuntu 13.3.0-6ubuntu2 24.04.1) 13.3.0
Compilação sequencial	gcc O3 Wall Wextra stdc11 fopenmp o fire_seq fire_seq.c fopenmp gcc O3 Wall Wextra stdc11 - fopenmp o fire_omp fire_omp.c fopenmp
Compilação paralela	mesmos parâmetros de compilação (inclusive -O3), alvo fire_omp (fire_omp.c) – os dois binários recebem o mesmo nível de otimização, incluindo a vetorização automática (SIMD) que o compilador aplica a ambos
Execução OpenMP	T lido do arquivo de entrada e fixado com <code>num_threads(T)</code> e <code>omp_set_dynamic(0)</code> ; a política de escalonamento do laço de atualização é fixada em tempo de compilação na diretiva <code>omp</code> <code>for</code> – duas políticas foram medidas no Linux (<code>guided(1024)</code> e <code>dynamic(1024)</code>), cada uma recompilando <code>fire_omp</code> isolada- mente; a versão final do código usa <code>guided(1024)</code>
Versão dos fontes	branch <code>main</code> , com a fusão de barreiras, o pré-cálculo dos pesos dos vizinhos e a padronização dos índices para <code>long long</code> descritos na Seção 3.7 e adiante

7.2 Cargas e protocolo de execução

Tabela 7: Cargas fornecidas. T indica o valor original, que pode variar nos experimentos.

Carga	L	C	P	T	F	Z
Pequena	400	500	80	4	3	2
Média	1200	1500	100	8	6	4
Grande	2500	2500	100	8	10	6

Cada configuração foi executada pelo script `scripts/run_macos.sh`: a versão sequencial é executada uma vez por repetição (sem variar threads) e a versão paralela é executada para $T \in \{2, 4, 6, 8\}$ (valores padrão do script), com o 4º campo da primeira linha da entrada sobrescrito temporariamente a cada valor de T testado; os demais parâmetros da Tabela acima são mantidos. Foram realizadas 10 repetições por combinação de carga e T , totalizando $3 \times (10 + 4 \times 10) = 150$ execuções por rodada. O script foi executado uma vez para cada uma das três políticas de escalonamento comparadas (`guided(1024)`, `static` e `dynamic(1024)`), recompilando `fire_omp` a cada rodada apenas com o argumento de `schedule` trocado na diretiva `omp for` (Seção 5.3), totalizando $3 \times 150 = 450$ execuções no macOS. No Linux, `guided(1024)` e `dynamic(1024)` foram executadas cada uma em uma rodada de 150 execuções (10 repetições por combinação), totalizando 300; `static` não foi medida nesse ambiente.

7.3 Delimitação do trecho cronometrado

O recorte especificado inclui a ativação das zonas, a propagação, a atualização dos estados e tempos, as estatísticas, a troca das matrizes e a verificação da parada. Exclui a leitura, a validação, a alocação, a geração da floresta, a definição dos focos, a construção do mapa, o checksum, os percentuais e a impressão. A medição deve utilizar `omp_get_wtime()`.

7.4 Métricas

Para a mesma carga, o speedup e a eficiência percentual são definidos por

$$S(T) = \frac{t_{\text{seq}}}{t_{\text{omp}}(T)}, \quad E(T) = \frac{S(T)}{T} \times 100\%. \quad (7)$$

A referência é o programa sequencial.

8 Resultados

8.1 MacOS

Os resultados desta seção foram obtidos no ambiente macOS descrito na Seção 7, comparando as três políticas de escalonamento do laço de atualização discutidas na Seção 5.3: `guided(1024)` (a escolhida para a versão final de `fire_omp.c`), `static` e `dynamic(1024)`. Cada política foi medida em uma rodada independente do script `scripts/run_macos.sh`, com o mesmo binário recompilado apenas trocando o argumento de `schedule` na diretiva `omp for`; t_{seq} é a média das 30 execuções sequenciais (10 por rodada) para aquela carga, e t_{omp} é a média de 10 repetições por combinação de política e T . Nesta rodada, `fire_seq` e `fire_omp` foram compilados com os mesmos parâmetros de otimização (`-O3` nos dois), de modo que $S(T) = t_{\text{seq}}/t_{\text{omp}}(T)$ isola o ganho da paralelização entre threads: a vetorização automática (SIMD) que o compilador aplica dentro de cada thread já está presente na referência sequencial, não apenas na versão OpenMP. A Seção 8.3 mostra o que acontece com essas mesmas métricas quando esse cuidado é relaxado.

Tabela 8: Tempos e métricas de desempenho — carga pequena (macOS, Apple M4), comparando as três políticas. Ambos os binários compilados com `-O3`.

Schedule	T	t_{seq} (s)	t_{omp} (s)	$S(T)$	$E(T)$ (%)
dynamic(1024)	2	0.077993	0.045236	1.72	86.21
	4	0.077993	0.025682	3.04	75.92
	6	0.077993	0.023158	3.37	56.13
	8	0.077993	0.021404	3.64	45.55
guided(1024)	2	0.077993	0.047567	1.64	81.98
	4	0.077993	0.025740	3.03	75.75
	6	0.077993	0.023512	3.32	55.29
	8	0.077993	0.022394	3.48	43.53
static	2	0.077993	0.046504	1.68	83.86
	4	0.077993	0.025160	3.10	77.50
	6	0.077993	0.026746	2.92	48.60
	8	0.077993	0.022774	3.42	42.81

Tabela 9: Tempos e métricas de desempenho — carga média (macOS, Apple M4), comparando as três políticas. Ambos os binários compilados com `-O3`.

Schedule	T	t_{seq} (s)	t_{omp} (s)	$S(T)$	$E(T)$ (%)
dynamic(1024)	2	0.861671	0.517304	1.67	83.28
	4	0.861671	0.317482	2.71	67.85
	6	0.861671	0.258590	3.33	55.54
	8	0.861671	0.216119	3.99	49.84
guided(1024)	2	0.861671	0.531620	1.62	81.04
	4	0.861671	0.329077	2.62	65.46
	6	0.861671	0.272585	3.16	52.69
	8	0.861671	0.228740	3.77	47.09
static	2	0.861671	0.489486	1.76	88.02
	4	0.861671	0.277753	3.10	77.56
	6	0.861671	0.276854	3.11	51.87
	8	0.861671	0.226592	3.80	47.53

Tabela 10: Tempos e métricas de desempenho — carga grande (macOS, Apple M4), comparando as três políticas. Ambos os binários compilados com `-O3`.

Schedule	T	t_{seq} (s)	t_{omp} (s)	$S(T)$	$E(T)$ (%)
dynamic(1024)	2	2.994227	1.841901	1.63	81.28
	4	2.994227	1.181183	2.54	63.37
	6	2.994227	0.972402	3.08	51.32
	8	2.994227	0.807000	3.71	46.38
guided(1024)	2	2.994227	1.893864	1.58	79.05
	4	2.994227	1.206397	2.48	62.05
	6	2.994227	0.985366	3.04	50.64
	8	2.994227	0.801603	3.73	46.69
static	2	2.994227	1.731216	1.73	86.48
	4	2.994227	1.123987	2.66	66.60
	6	2.994227	1.059621	2.83	47.10
	8	2.994227	0.887220	3.38	42.19

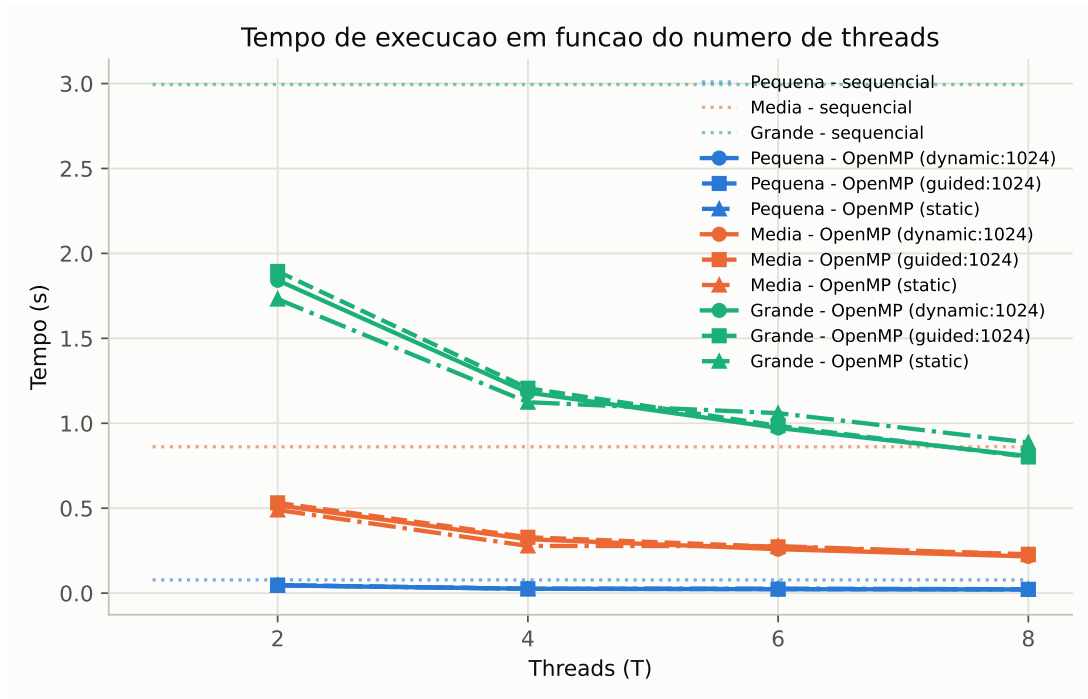


Figura 1: Tempo em função do número de threads, para as três políticas de escalonamento e as três cargas (cor = carga, traço e marcador = schedule).

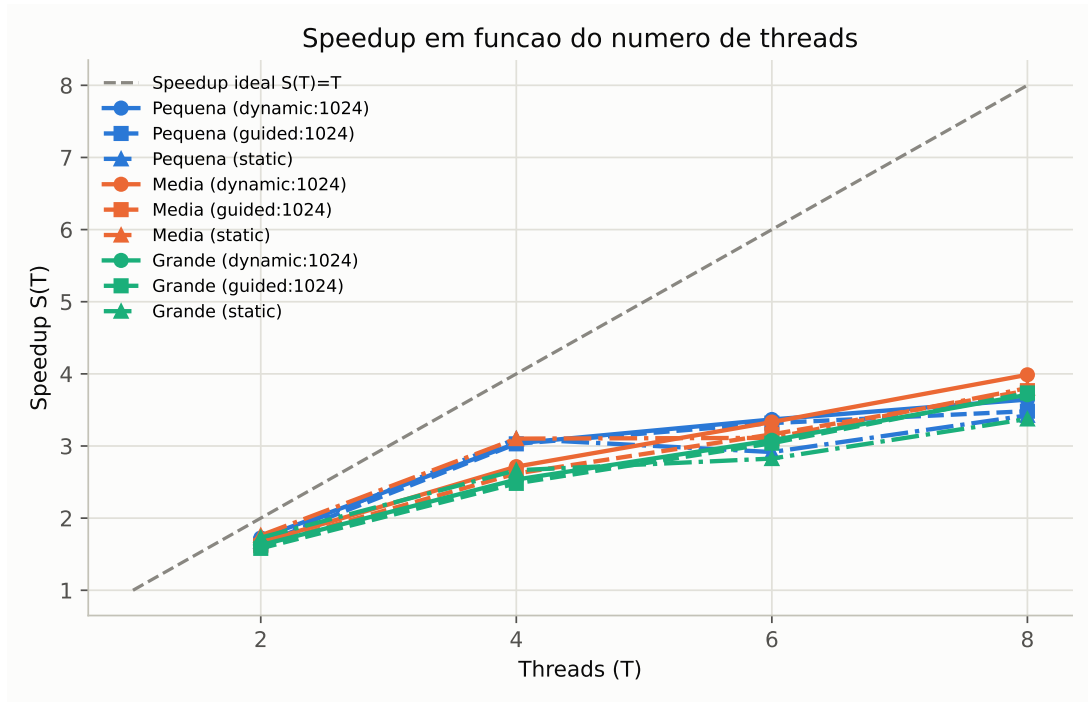


Figura 2: Speedup em função do número de threads, com referência ideal $S(T) = T$.

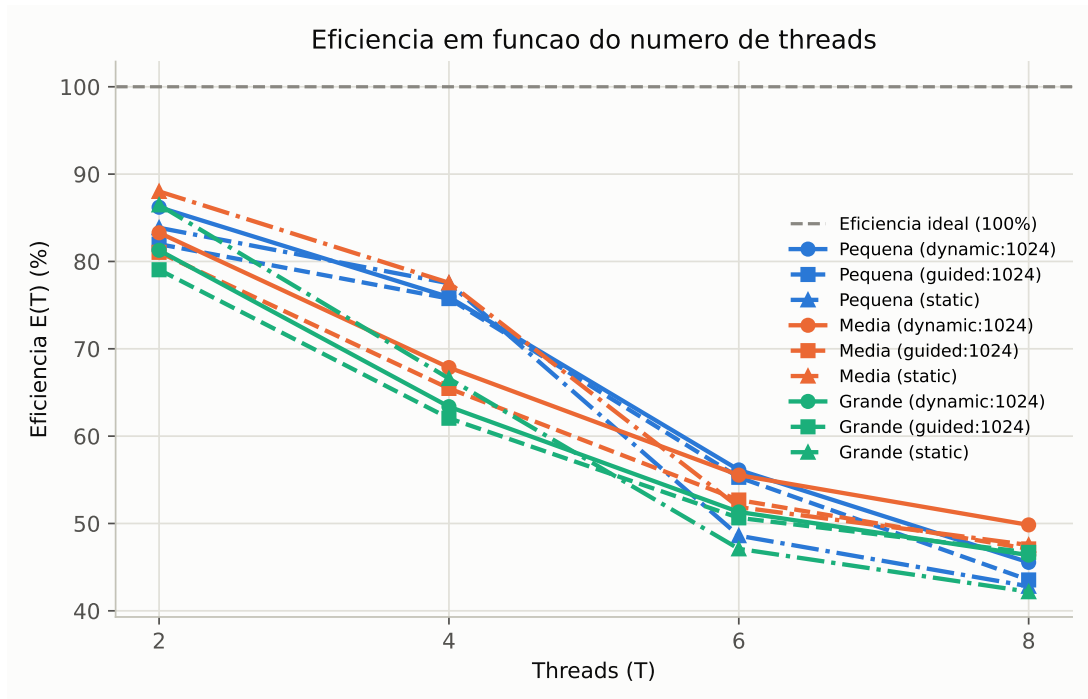


Figura 3: Eficiência percentual em função do número de threads.

8.2 Linux

Os resultados desta seção foram obtidos no ambiente Linux descrito na Seção 7. Neste ambiente foram medidas duas políticas de escalonamento, `guided(1024)` (a escolhida para a versão final de `fire_omp.c`) e `dynamic(1024)`, cada uma em uma rodada independente

com 10 repetições por combinação de carga e $T \in \{2, 4, 6, 8\}$ (mais 10 execuções sequenciais por carga em cada rodada). A política **static** não foi testada no Linux. Para que as duas políticas usem a mesma referência, t_{seq} é a média das 20 execuções sequenciais de cada carga (10 por rodada), e t_{omp} é a média das 10 repetições de cada política e T . Os dois binários foram compilados com `-O3`, e o *checksum* e o número de passos foram idênticos em todas as execuções de uma mesma carga, sequenciais e paralelas, nas duas rodadas.

Tabela 11: Tempos e métricas de desempenho — carga pequena (Linux, AMD Ryzen 5), comparando **dynamic(1024)** e **guided(1024)** (médias de 10 repetições). Ambos os binários compilados com `-O3`.

Schedule	T	t_{seq} (s)	t_{omp} (s)	$S(T)$	$E(T)$ (%)
dynamic(1024)	2	0.264346	0.183563	1.44	72.00
	4	0.264346	0.095877	2.76	68.93
	6	0.264346	0.086613	3.05	50.87
	8	0.264346	0.079575	3.32	41.52
guided(1024)	2	0.264346	0.182449	1.45	72.44
	4	0.264346	0.097306	2.72	67.92
	6	0.264346	0.091981	2.87	47.90
	8	0.264346	0.081095	3.26	40.75

Tabela 12: Tempos e métricas de desempenho — carga média (Linux, AMD Ryzen 5), comparando **dynamic(1024)** e **guided(1024)** (médias de 10 repetições). Ambos os binários compilados com `-O3`.

Schedule	T	t_{seq} (s)	t_{omp} (s)	$S(T)$	$E(T)$ (%)
dynamic(1024)	2	3.087972	2.135249	1.45	72.31
	4	3.087972	1.129446	2.73	68.35
	6	3.087972	1.019873	3.03	50.46
	8	3.087972	0.941679	3.28	40.99
guided(1024)	2	3.087972	2.133405	1.45	72.37
	4	3.087972	1.126572	2.74	68.53
	6	3.087972	1.112176	2.78	46.28
	8	3.087972	0.949525	3.25	40.65

Tabela 13: Tempos e métricas de desempenho — carga grande (Linux, AMD Ryzen 5), comparando `dynamic(1024)` e `guided(1024)` (médias de 10 repetições). Ambos os binários compilados com `-O3`.

Schedule	T	t_{seq} (s)	t_{omp} (s)	$S(T)$	$E(T)$ (%)
dynamic(1024)	2	10.755516	7.827994	1.37	68.70
	4	10.755516	4.503688	2.39	59.70
	6	10.755516	3.634359	2.96	49.32
	8	10.755516	3.373458	3.19	39.85
guided(1024)	2	10.755516	7.444430	1.44	72.24
	4	10.755516	3.931087	2.74	68.40
	6	10.755516	3.842922	2.80	46.65
	8	10.755516	3.360784	3.20	40.00

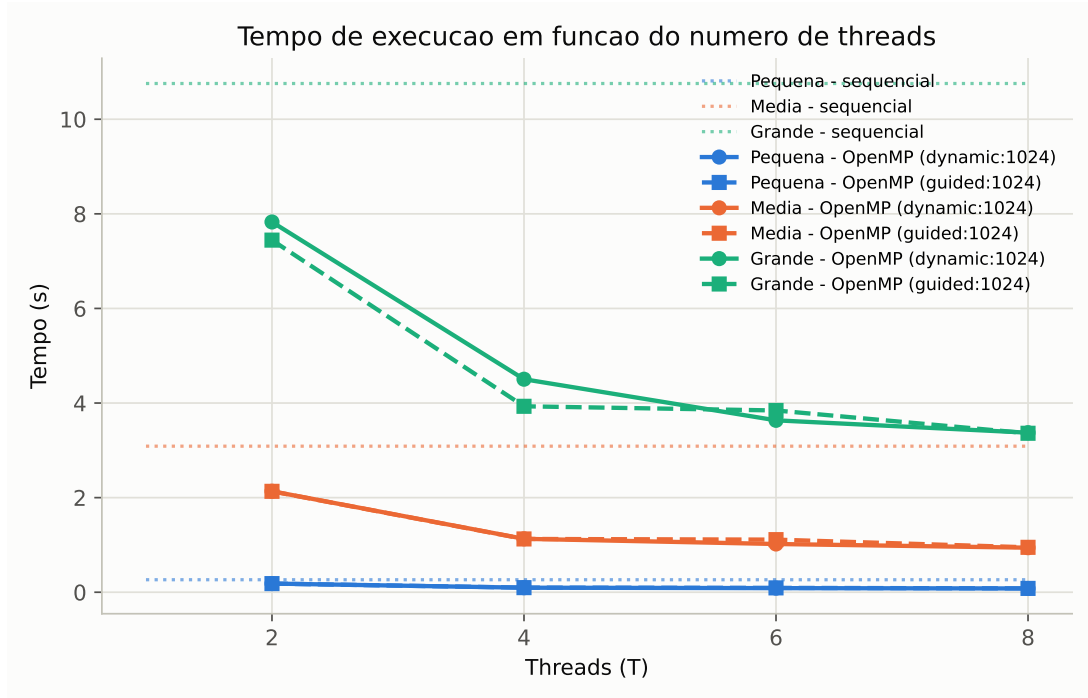


Figura 4: Tempo em função do número de threads, para as duas políticas e as três cargas (Linux; cor = carga, traço e marcador = schedule).

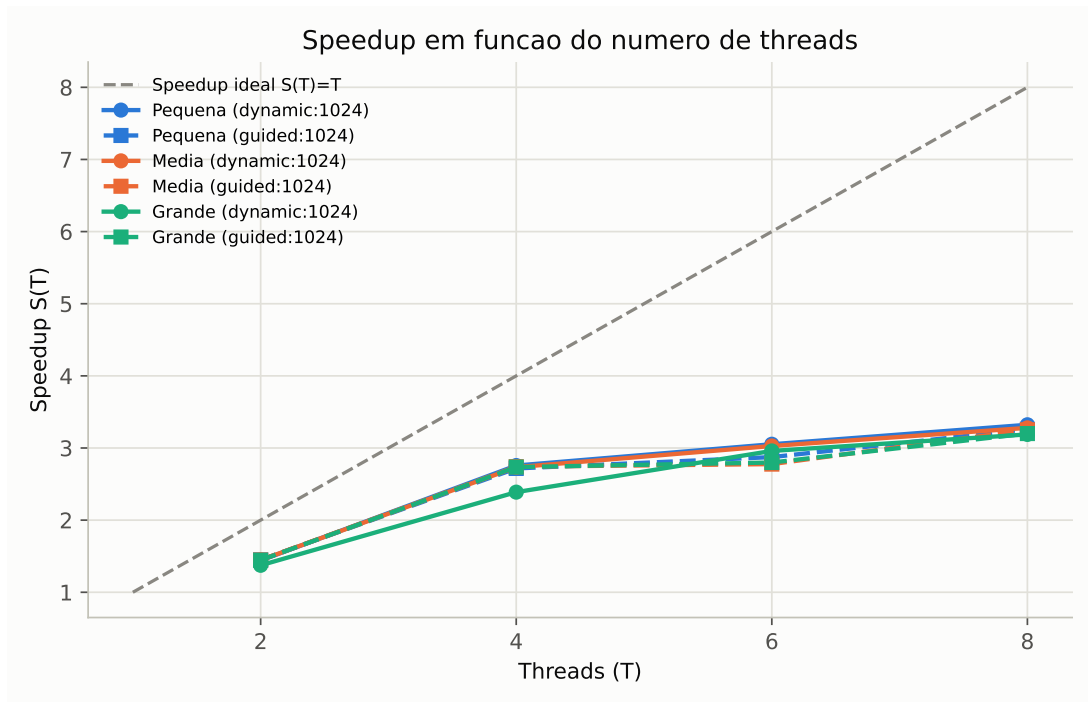


Figura 5: Speedup em função do número de threads, com referência ideal $S(T) = T$ (Linux).

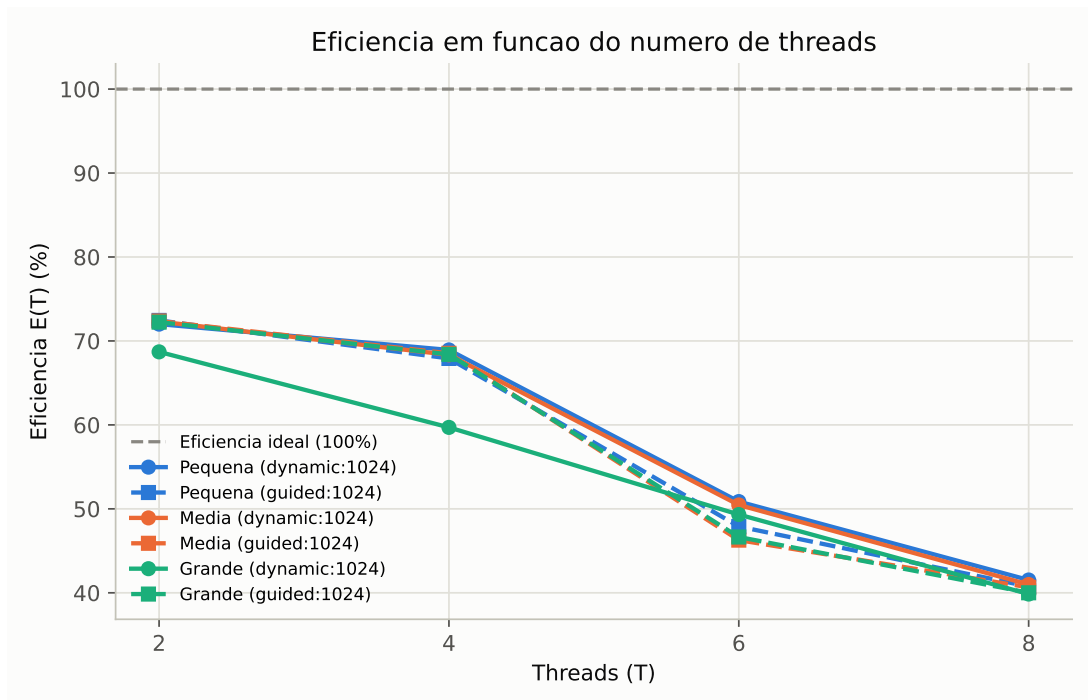


Figura 6: Eficiência percentual em função do número de threads (Linux).

8.3 Efeito da vetorização automática (SIMD) na linha de base sequencial

A compilação com `-O3` habilita, além de outras otimizações, a vetorização automática (SIMD) dos laços de `fire_seq.c` e de `fire_omp.c` – sem que uma única linha de código precise mudar para isso. Isso levanta uma questão metodológica: comparar a versão OpenMP contra uma referência sequencial compilada *sem* otimização (`-O0`, "como se não soubéssemos que o processador tem uma unidade SIMD") mede uma coisa diferente de compará-la contra uma referência igualmente vetorizada. A primeira comparação, aqui chamada de **ingênua**, mistura dois tipos de paralelismo: o paralelismo de dados dentro de uma única thread (SIMD, obtido de graça ao trocar `-O0` por `-O3`) e o paralelismo entre threads via OpenMP, que é o que este trabalho de fato estuda. A segunda, chamada de **justa** e usada nas tabelas e nos gráficos da seção anterior, isola o segundo ao compilar as duas versões com os mesmos parâmetros de otimização.

Para quantificar a diferença, `fire_seq` foi recompilado uma vez sem nenhuma flag de otimização, gerando uma segunda referência sequencial "pura". A Tabela 14 mostra o ganho desse único fator – vetorização automática, sem nenhum paralelismo entre threads – isoladamente.

Tabela 14: Ganho isolado da vetorização automática (`-O0`→`-O3`), com uma única thread em ambos os casos.

Carga	$t_{\text{seq}} \text{ -O0 (s)}$	$t_{\text{seq}} \text{ -O3 (s)}$	Razão
Pequena	0.297597	0.077993	$3.82\times$
Média	3.437358	0.861671	$3.99\times$
Grande	11.986134	2.994227	$4.00\times$

O resultado é notavelmente consistente entre as três cargas: a vetorização automática, sozinha, já produz um ganho de $\approx 4\times$ – praticamente do mesmo tamanho que o ganho de *todo* o paralelismo entre threads medido na Seção anterior em $T = 8$ ($S_{\text{justo}}(8)$ entre $3.48\times$ e $3.99\times$, Tabelas anteriores). Ou seja: se a linha de base sequencial não for vetorizada, metade (ou mais) do "speedup" atribuído à versão OpenMP na verdade não vem de paralelizar entre threads, e sim de uma otimização de compilador que o programa sequencial também poderia ter recebido de graça.

A Tabela 15 repete as métricas de `guided(1024)` (a política final) das três cargas, lado a lado nas duas comparações – ambas usam exatamente os mesmos $t_{\text{omp}}(T)$ das tabelas da seção anterior, só trocando o denominador t_{seq} .

Tabela 15: Comparação ingênua (vs. `fire_seq -00`) e justa (vs. `fire_seq -03`) para `guided(1024)`.

Carga	T	$S_{\text{ingênuo}}(T)$	$E_{\text{ingênuo}}(T)$ (%)	$S_{\text{justo}}(T)$	$E_{\text{justo}}(T)$ (%)
Pequena	2	6.62	331.1	1.64	82.0
	4	12.27	306.6	3.03	75.8
	6	13.41	223.5	3.32	55.3
	8	14.69	183.7	3.48	43.5
Média	2	6.70	335.2	1.62	81.0
	4	10.69	267.2	2.62	65.5
	6	13.18	219.6	3.16	52.7
	8	15.65	195.6	3.77	47.1
Grande	2	6.61	330.3	1.58	79.0
	4	10.20	255.1	2.48	62.0
	6	12.36	206.0	3.04	50.6
	8	14.89	186.1	3.73	46.7

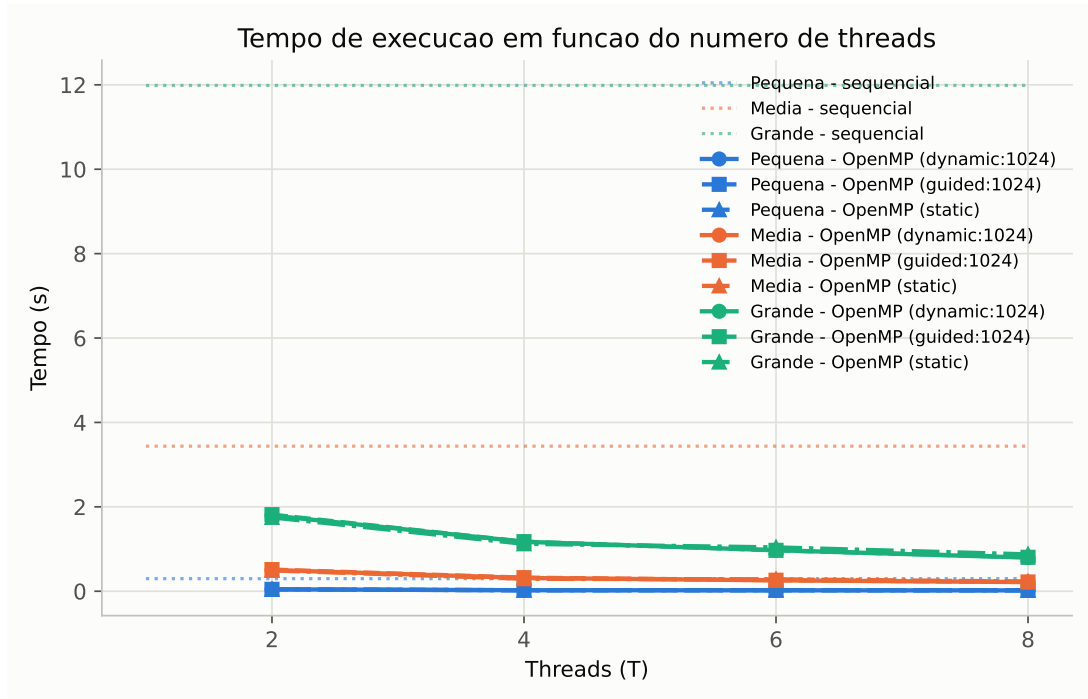


Figura 7: Tempo em função do número de threads (comparação ingênua: `fire_omp -03` contra `fire_seq` sem otimização).

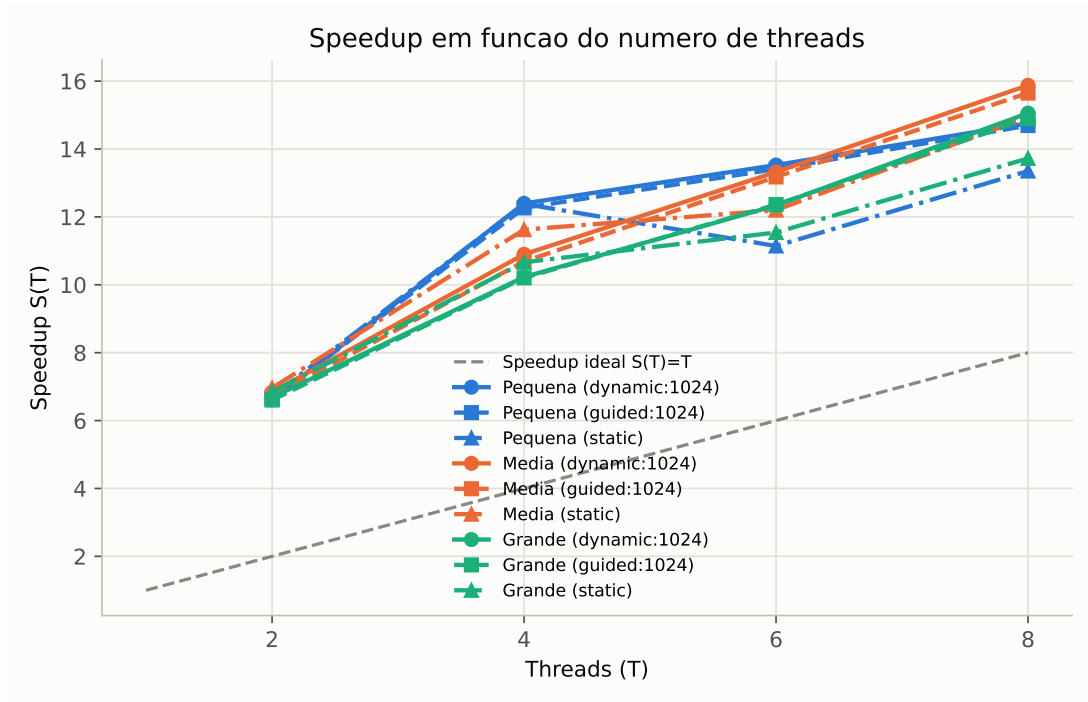


Figura 8: Speedup ingênuo em função do número de threads – note a escala do eixo Y, bem acima da referência ideal $S(T) = T$.

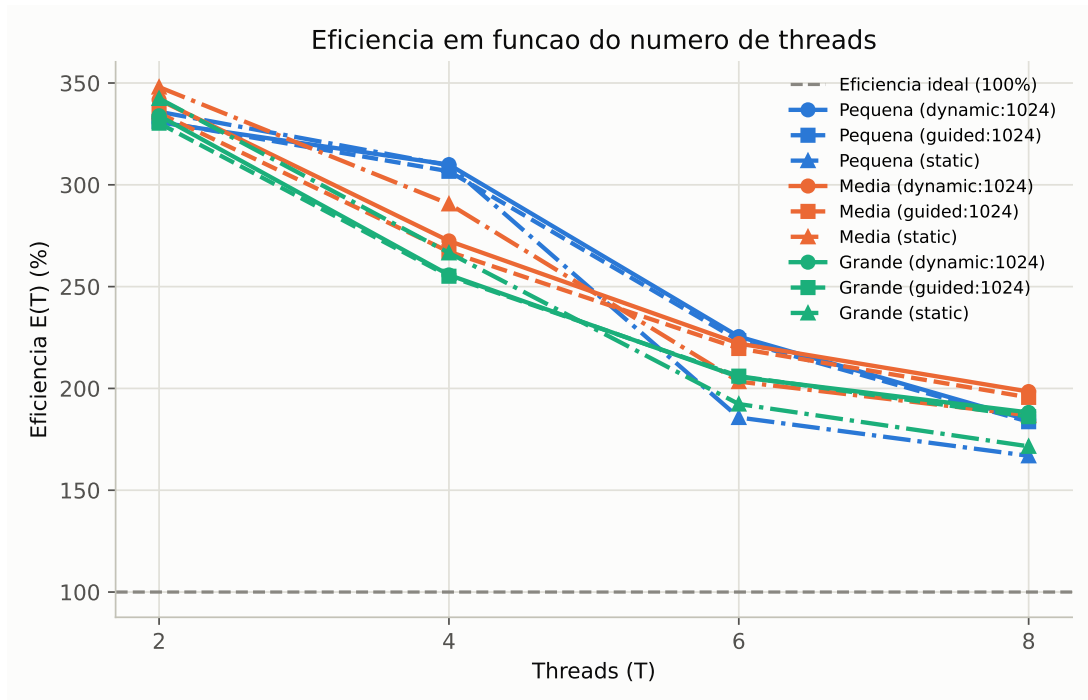


Figura 9: Eficiência ingênuo em função do número de threads – valores acima de 100% indicam que o "speedup" medido não é apenas paralelismo entre threads.

A diferença é grande o bastante para mudar a leitura qualitativa dos resultados. Na comparação ingênuo, $E_{\text{ingênuo}}(2)$ já passa de 330% nas três cargas. Esse "excesso" de eficiência é inteiramente explicado pela Tabela 14: dividir $E_{\text{ingênuo}}(T)$ pela razão $\approx 4\times$

da vetorização isolada recupera, com boa aproximação, $E_{\text{justo}}(T)$ (por exemplo, na carga média, $335,2\%/3,99 \approx 84\%$, próximo dos $81,0\%$ medidos diretamente – a pequena diferença vem do próprio ruído de medição entre rodadas independentes). Isso confirma que os dois fatores são aproximadamente independentes e multiplicativos: $\text{speedup total} \approx \text{speedup SIMD}$ (por thread) $\times$ speedup de paralelismo (entre threads).

Por esse motivo, a comparação **justa** (Tabelas da Seção 8 e Tabela 15, colunas $S_{\text{justo}}/E_{\text{justo}}$) é a usada como referência principal deste relatório e nas Seções 8 e seguintes: é ela que isola a pergunta que o enunciado pede para responder, o quanto a *paralelização entre threads* melhora o desempenho.

Vale lembrar que a utilização de `-O3` também pode ser lida como uma paralelização implícita, mas entre dados (SIMD) e não entre threads. Por utilizarmos explicitamente a diretiva `omp simd` em trechos vetorizáveis do código, achamos interessante trazer a comparação entre um código sequencial compilado sem a flag de otimização e o código paralelo compilado com a flag de otimização, para mostrar a capacidade de melhora de desempenho que a vetorização automática do compilador traz, mesmo sem a utilização de threads.

Salvo indicação em contrário, as observações abaixo se referem ao ambiente macOS descrito na Seção 7. No ambiente Linux (Seção 8.2) foram medidas `guided(1024)` e `dynamic(1024)`, com 10 repetições por ponto; a análise correspondente está no parágrafo *Resultados no Linux*, ao final desta seção.

Efeito da fusão de barreiras e do pré-cálculo dos pesos. Uma rodada anterior deste relatório, no mesmo ambiente e com o mesmo script, media a versão com `ativar_zonas` ainda separada (3 barreiras por passo), os pesos P_v recalculados a cada vizinho e apenas a política `guided(1024)`. Comparando os mesmos pontos (carga, T) dessa política entre as duas rodadas, o ganho aparece em todas as combinações medidas: na carga grande, $S(8)$ sobe de 2.79 para 3.73 ($E(8)$ de 34.93% para 46.69%); na carga média, de $S(8) = 3.38$ para 3.77 ($E(8)$ de 42.23% para 47.09%); na carga pequena, de $S(4) = 2.59$ para 3.03. O ganho relativo tende a crescer com T , como seria de se esperar de uma otimização que remove um custo fixo por passo (uma barreira inteira): esse custo pesa proporcionalmente mais quanto mais threads precisam ser sincronizadas a cada passo.

Ganho de threads limitado pelo número de núcleos de desempenho. O Apple M4 utilizado possui apenas 4 núcleos de desempenho e 6 de eficiência, sem SMT. Para as três cargas e as três políticas, $E(T)$ cai de forma monótona à medida que T cresce, de 79–88% em $T = 2$ para 42–50% em $T = 8$. O speedup continua crescendo em todo o intervalo medido para quase todas as combinações, mas a taxas decrescentes: na carga média com `guided(1024)`, por exemplo, $S(T)$ sobe de 1.62 em $T = 2$ para 3.77 em $T = 8$ – um fator de 2.33 ao quadruplicar o número de threads, bem abaixo do fator 4 que um ganho ideal teria. Esse achatamento é compatível com threads acima de 4 sendo atendidas, ao menos em parte, por núcleos de eficiência mais lentos que os de desempenho, e se repete nas três políticas de escalonamento testadas.

static é consistentemente a pior política em T alto; a diferença entre `guided(1024)` e `dynamic(1024)` não é robusta entre rodadas. Em $T = 6$ e $T = 8$, `static` perde eficiência de forma acentuada nas três cargas (42–49% em $T = 8$, contra 43–50% de `guided(1024)/dynamic(1024)` no mesmo ponto), repetindo o padrão já visto na rodada anterior (com os flags de compilação ainda inconsistentes) e confirmando a hipótese da Seção 5.3: com poucas threads, os blocos contíguos e fixos de `static` ainda

cobrem uma mistura razoável de células intactas e não intactas, mas conforme T cresce, os blocos ficam menores e mais sensíveis a onde a frente de fogo está concentrada naquele passo, e o desbalanceamento resultante deixa de ser compensado pelo overhead (que **static** praticamente não tem). Esse resultado é robusto: reproduziu-se em ambos os ambientes das rodadas independentes. Já a diferença entre **guided(1024)** e **dynamic(1024)** *não* se reproduziu: nesta rodada, no cluster, **dynamic(1024)** fica à frente de **guided(1024)** em praticamente todos os pontos medidos (por exemplo, $S(8) = 3.99$ contra 3.77 na carga média, e $S(8) = 3.64$ contra 3.48 na pequena), com exceção de um empate técnico na carga grande em $T = 8$ ($S(8) = 3.73$ de **guided(1024)** contra 3.71 de **dynamic(1024)**, diferença de 0.02) – o oposto do que uma rodada anterior deste relatório havia registrado, na qual **guided(1024)** vencia com folga nas cargas média e grande em $T = 8$. Como a diferença entre as duas políticas fica quase sempre abaixo de 10%, parte dessa inversão provavelmente vem do ruído de medição entre sessões independentes (variação térmica, outros processos em segundo plano), não de uma vantagem estrutural de uma política sobre a outra. **guided(1024)** foi mantida como a política final do código por nunca perder feio para **dynamic(1024)** (a maior desvantagem observada é de $\approx 7\%$ em $S(T)$) e por nunca perder para **static**; mas uma conclusão mais forte entre **guided(1024)** e **dynamic(1024)** exigiria mais rodadas completas do que as disponíveis nesta versão do relatório – idealmente em outro ambiente. O Linux (Seção 8.2) traz uma segunda comparação entre essas duas políticas, com 10 repetições por ponto em cada uma, embora em rodadas separadas e sem **static**.

Resultados no cluster Linux. No ambiente Linux (AMD Ryzen 5, 8 núcleos, Seção 8.2), com ambos os binários em -O3, as duas políticas seguem o mesmo padrão geral: o speedup cresce com T nas três cargas e a eficiência cai de $\approx 72\%$ em $T = 2$ para 40–42% em $T = 8$, com $S(8)$ entre 3.19 e 3.32. Excluídos os pontos com *outliers* discutidos a seguir, as três cargas se comportam de forma praticamente idêntica: para cada T e política, os speedups diferem em no máximo 0.13 entre pequena, média e grande, ou seja, o ganho por paralelização não depende do tamanho da carga nos tamanhos testados.

Na carga grande, **dynamic(1024)** apresenta *outliers* em $T = 2$ (repetições 4 e 5: 8.76 s e 8.73 s, contra 7.52–7.68 s nas demais) e em $T = 4$ (repetições 2, 3 e 4: 6.00 s, 6.61 s e 4.74 s, contra 3.93–3.97 s nas outras sete). Isso eleva as médias (7.828 s e 4.504 s; coeficientes de variação de 6% e 22%) e produz os valores atípicos $S(2) = 1.37$ e $S(4) = 2.39$ da tabela; pelas medianas (7.656 s e 3.963 s) os valores seriam $S(2) = 1.40$ e $S(4) = 2.71$, alinhados com o restante. A concentração em poucas repetições sugere interferência transitória externa (outros processos na máquina) e não um efeito da política, mas isso não foi verificado.

A diferença entre as políticas aparece em $T = 6$. Com **dynamic(1024)**, $S(6)$ fica entre 2.96 e 3.05 nas três cargas, contra 2.78–2.87 de **guided(1024)** (vantagem de 6 a 9%), e a eficiência entre 49 e 51%, contra 46–48%. A diferença é consistente dentro das rodadas: em cada carga, a repetição mais lenta de **dynamic(1024)** em $T = 6$ (3.649 s, 1.026 s e 0.0868 s nas cargas grande, média e pequena) é mais rápida que a mais rápida de **guided(1024)** (3.793 s, 1.083 s e 0.0897 s). O platô entre $T = 4$ e $T = 6$ é, por isso, pronunciado sob **guided(1024)** (o tempo cai de 1 a 6%: por exemplo, de 1.127 s para 1.112 s na carga média) e menor sob **dynamic(1024)** (8 a 10%, usando a mediana na carga grande), embora em ambos bem abaixo da queda de 33% que um ganho ideal de 4 para 6 threads teria. Em $T = 8$ as duas políticas praticamente empatam ($S(8)$ de 3.19 contra 3.20 na grande, 3.28 contra 3.25 na média e 3.32 contra 3.26 na pequena, diferenças de até

2%).

Esse resultado deve ser lido com cautela: as duas políticas foram medidas em rodadas separadas, então a variação entre sessões pode contribuir para a diferença em $T = 6$, e no macOS a ordem entre as duas políticas já se inverteu entre rodadas. A direção observada coincide com a da rodada mais recente do macOS (`dynamic(1024)` à frente em T alto), mas aqui a vantagem se concentra em $T = 6$, e não em $T = 8$. Vale notar que a desvantagem de `guided(1024)` em $T = 6$ chega a $\approx 8\%$ em $S(T)$ na carga média (2.78 contra 3.03), acima dos $\approx 7\%$ citados acima como pior caso no macOS. Em relação ao macOS, o padrão qualitativo é o mesmo (eficiência decrescente e ganho sublinear em $T = 8$), mas o speedup em $T = 8$ é menor no Linux (cerca de 3.2–3.3, contra 3.48–3.99 para as mesmas duas políticas no macOS) e a eficiência em $T = 2$ também ($\approx 72\%$, contra 79–86%). Os tempos absolutos não são comparáveis entre os ambientes, pois as máquinas, os compiladores e os sistemas diferem; a comparação vale apenas para $S(T)$ e $E(T)$. Este trabalho não isola a causa do platô, e por isso registra o comportamento sem atribuí-lo a um mecanismo específico.

Consistência dos resultados. Como reportado na Seção 6, o checksum e a quantidade de passos executados foram idênticos entre `fire_seq` e `fire_omp` para $T \in \{2, 4, 6, 8\}$, nas três cargas e nas três políticas de escalonamento, confirmando que nem a paralelização, nem a escolha de schedule, nem as otimizações desta rodada (fusão de barreiras, pesos pré-calculados, índices `long long`) alteram o resultado da simulação, apenas o seu tempo de execução, exatamente como esperado da modelagem com atualização síncrona e reduções (Seção 3.7).

9 Conclusão

As implementações sequencial e paralela atendem aos requisitos do enunciado: mesma saída determinística para uma dada entrada, independentemente da quantidade de threads e da política de escalonamento, com a versão paralela construída sobre uma região `parallel` persistente, um único `omp for` por passo (após a fusão da ativação de zonas com a atualização de células) com `default(none)`, reduções para os contadores e SIMD no cálculo do potencial de ignição, cujos pesos por direção são pré-calculados uma única vez fora do trecho cronometrado. No ambiente macOS avaliado (Apple M4, 10 núcleos), três políticas de escalonamento foram medidas para o laço de atualização – `guided(1024)`, `static` e `dynamic(1024)` –, satisfazendo a comparação de pelo menos duas políticas exigida pelo enunciado, com `fire_seq` e `fire_omp` compilados com os mesmos parâmetros de otimização (`-O3`) para que o speedup medido isole o ganho de paralelização entre threads (Seção 8.3). `static` foi consistentemente a pior política a partir de $T = 6$ em duas rodadas de medição independentes, enquanto `guided(1024)` e `dynamic(1024)` sustentaram melhor o speedup em T alto; a vantagem entre essas duas últimas não se mostrou robusta entre rodadas, então `guided(1024)` foi mantida como política final. Com `guided(1024)`, a versão OpenMP produziu speedups entre $1.58\times$ e $3.77\times$ em relação à versão sequencial igualmente otimizada (para $T \in \{2, 4, 6, 8\}$), com eficiência decrescente à medida que a quantidade de threads ultrapassa o número de núcleos de desempenho da máquina (de 79–88% em $T = 2$ para 42–50% em $T = 8$, todas as políticas). A diferença entre a carga grande e a média, que já vinha diminuindo ao longo das otimizações deste trabalho, ficou dentro do ruído de medição especificamente para `guided(1024)` ($S(8) = 3.73$ contra 3.77),

embora ainda mensurável sob `dynamic(1024)` e `static`.

No ambiente Linux avaliado (AMD Ryzen 5, 8 núcleos), foram medidas `guided(1024)` e `dynamic(1024)`, cada uma com 10 repetições por ponto, também com ambos os binários em `-O3` e com `checksum` idêntico em todas as execuções de cada carga. A versão OpenMP atingiu speedups de até $3.32\times$ em $T = 8$ (eficiência de 40–42%, contra $\approx 72\%$ em $T = 2$), praticamente sem diferença entre as três cargas. As duas políticas empataram em $T = 8$ e em $T \leq 4$ (exceto por *outliers* de `dynamic(1024)` na carga grande), mas em $T = 6$ `dynamic(1024)` ficou 6 a 9% à frente ($S(6) \approx 3.0$ contra 2.78–2.87), diferença consistente entre as repetições, com `guided(1024)` apresentando um platô entre $T = 4$ e $T = 6$.

Um achado deste trabalho, tratado em detalhe na Seção 8.3, é de natureza metodológica: comparar a versão OpenMP (compilada com `-O3`) contra uma referência sequencial sem otimização produz speedups "ingênuos" de até $15.9\times$ e eficiências de até 348% – números sem sentido físico para paralelismo entre threads, que se explicam quase inteiramente pelo ganho de $\approx 4\times$ que a vetorização automática (SIMD) do compilador já dá, sozinha, a um programa de uma única thread. Manter os mesmos parâmetros de otimização nas duas versões, como feito nesta rodada, é o que permite atribuir o speedup medido especificamente à paralelização entre threads, e não a uma mistura de dois fenômenos distintos.

Referências

- [1] Universidade de São Paulo, ICMC/SSC. *SSC0903 — Primeiro Trabalho Prático: simulação paralela da propagação direcional de incêndio com zonas de contenção*. Arquivo `Trab01-Fire-OMP-v1.pdf`, setembro de 2026.